

Human-Centric Artificial Intelligence Project-3 Report

Name – Surname: Barış Kaplan

Matriculation Number: 669584

In this project, the following class mappings are used:

Class 0 => "World"

Class 1 => "Sports"

Class 2 => "Business"

Class 3 => "Sci/Tech"

Task 1:

Used Classification Model: Logistic Regression

Classifier Baseline Test Accuracy Achieved: 90.55%

```
# Split the imported data into train and test parts
self.X_train_raw = dataset['train']['text']
self.y_train = np.array(dataset['train']['label'])
self.X_test_raw = dataset['test']['text']
self.y_test = np.array(dataset['test']['label'])

# Text Feature Extraction
self.vectorizer = TfidfVectorizer(max_features=5000, stop_words='english')
self.X_train = self.vectorizer.fit_transform(self.X_train_raw).toarray()
self.X_test = self.vectorizer.transform(self.X_test_raw).toarray()

# Baseline Classifier
self.baseline_model = LogisticRegression(C=0.4, max_iter=200)
self.baseline_model.fit(self.X_train, self.y_train)
self.baseline_test_preds = self.baseline_model.predict(self.X_test)
self.baseline_acc = float(accuracy_score(self.y_test, self.baseline_test_preds))
```

Code Screenshot 1: Code used for train-test splitting, text feature extraction, and baseline model training, and baseline model evaluation using accuracy score between the ground truth y_test data and baseline model test data predictions

Task 2:

Simulated Expert's Test Dataset Accuracy Achieved: 92.70% (Specialized expertise in Class 1 (Sports))

```
# Simulated Expert
self.expert_test_preds = self.simulate_expert_predict(self.y_test)
self.expert_acc = float(accuracy_score(self.y_test, self.expert_test_preds))

def simulate_expert_predict(self, y_true):
    """ Simulates bounded human expert decisions. """
    expert_preds = []
    for label in y_true:
        # Specific domain specialization
```

```

if label == 1:
    expert_preds.append(label)
else:
    if np.random.rand() < 0.90:
        expert_preds.append(label)
    else:
        remaining_classes = [c for c in range(4) if c != label]
        expert_preds.append(np.random.choice(remaining_classes))
return np.array(expert_preds)

```

Code Screenshot 2: Code implemented for the predictions of a simulated expert and evaluation of the expert using the accuracy score between the ground-truth y_{test} data and the expert predictions on the y_{test} data

```

def simulate_expert_predict(self, y_true):
    """Simulates bounded human expert decisions."""
    expert_preds = []
    for label in y_true:
        # Specific domain specialization
        if label == 1:
            expert_preds.append(label)
        else:
            if np.random.rand() < 0.90:
                expert_preds.append(label)
            else:
                remaining_classes = [c for c in range(4) if c != label]
                expert_preds.append(np.random.choice(remaining_classes))
    return np.array(expert_preds)

```

Code Screenshot 3: Code implemented for the predictions of the simulated expert

Strengths of the Simulated Expert on the Dataset

- 1. Domain Expertise in Specific Regions of the Input Space:** This simulated expert exhibits a deterministic behavior for Class 1, which is the “Sports” field. It suitably mimics the knowledge of real-world human experts who possess deep knowledge in a specific field (Class 1: Sports), instead of holding a generalized and/or broad expertise. Therefore, it will serve as a realistic and resource-efficient proxy substitute to evaluate cost-efficient (efficient in terms of time, computational complexity, and financial budget) routing and decision delegation mechanisms within the learning-to-defer framework.
- 2. High Accuracy Achieved on the Test Dataset:** The simulated expert has achieved a high test dataset accuracy of “92.70%”, which means it is a highly reliable ground-truth substitute.
- 3. Significant Reduction of Time, Effort, and Cost Needed for Manual Data Labeling:** This simulated expert can serve as a highly reliable proxy for a manual domain expert workforce on the ground-truth dataset; as a result, this approach will significantly reduce the time, effort, and cost required for manual data labeling.

Weaknesses of the Simulated Expert on the Dataset

- 1. Uniform Error Distribution:** The simulated expert makes mistakes %10 of his/her choices. A class for the given text is randomly and uniformly chosen from the remaining options, with equal probability for each class option. However, this is not the case for the decisions of real-life human experts. Human experts rarely guess uniformly and randomly. They are typically

```

else:
    if np.random.rand() < 0.90:
        expert_preds.append(label)
    else:
        remaining_classes = [c for c in range(4) if c != label]
        expert_preds.append(np.random.choice(remaining_classes))

```

Code Screenshot 4: Code implemented for the predictions of the simulated expert (Else case)

influenced by cognitive biases, systematic biases, patterned behavior, and social context, which makes them usually confuse specific classes sharing similar features.

2. Class Imbalance Problem: When human experts encounter an unclear text, they will inherently develop a high level of prior bias towards the classes with more dominance than the others. However, my simulated expert cannot observe the frequencies of the classes (labels). Therefore, its guessing behavior and following prediction errors do not change based on which classes are seen more often, which means the simulated expert is not affected by the class distribution.

3. Static Error Ratio: The error ratio is static and 10% for all remaining classes except Class 1, which are Class 0, Class 2, and Class 3. However, the human experts' data labeling performance and errors made in this process change a lot based on several factors, such as tiredness level, daytime, and changing expertise levels for the remaining classes.

4. No Contextual Knowledge of the Input X Values: Even though the simulated expert knows the ground-truth class labels (Class 0 => "World", Class 1 => "Sports", Class 2 => "Business", Class 3 => "Sci/Tech"), it does not know how the features of the input values differ from each other in terms of the length, number, meaning, complexity, and structure of the words in each input text.

5. No “I am completely unsure” Option: This expert is forced to select from the available labels in the error case. However, our expert in real-life might not be able to classify the input into the provided categories and put it into a category like "Unknown".

Strengths of the Trained Baseline Classifier and Text Feature Extraction

1. English Stop Words Removal: The English stop words that do carry a little significance are removed from the vocabulary and consideration of the TF-IDF Vectorizer algorithm. So that it can focus on the words that carry more importance in that specific sentence, which will bring a higher model accuracy score on the unseen test data.

2. Overfitting Avoidance Level: The baseline classifier model that I used adequately prevents overfitting with the inverse of the regularization parameter set to 0.4 ($C = 0.4$), which will make the algorithm avoid overfitting and put higher levels of regularization penalties, and the maximum number of iterations (epochs) set to 200, which might prevent earlier convergence.

3. Maximum Number of Text Features Limitation: In my implementation, I have set the “*max_features*” parameter in the “TF-IDF Vectorizer” algorithm to **5000**, which will discard words other than the top 5000 most frequent words in the unique words set. This might make the algorithm avoid overfitting more by avoiding less frequent and unimportant words such as “the” and “and”.

Weaknesses of the Trained Baseline Classifier and Text Feature Extraction

1. Context and Word Order Loss: I have used a TFIDF-Vectorizer algorithm to extract the text features, which only focuses on the unigrams (individual words). It does not focus on the order of the words in the sentence they are included in, or the context of that word in the sentence.

2. Semantic Meaning Loss: The “TF-IDF Vectorizer algorithm” loses the semantic meaning of the words, meaning they treat “great” and “greaaat” or “buy” and “purchase” as different words in a pair, which will lead to poorer model generalization capability and reduced performance on the unseen (out-of-the-vocabulary) data variations.

3. Maximum Number of Text Features Limitation: In my implementation, I have set the “*max_features*” parameter in the “TF-IDF Vectorizer” algorithm to **5000**, which will discard words other than the top 5000 most frequent words in the unique words set. Thus, this might limit the performance of the vectorizer algorithm if the dataset used has more than 5000 unique words and the words starting from the 5001st contain important words (which are not stop words) to be considered as well.

Task 3:

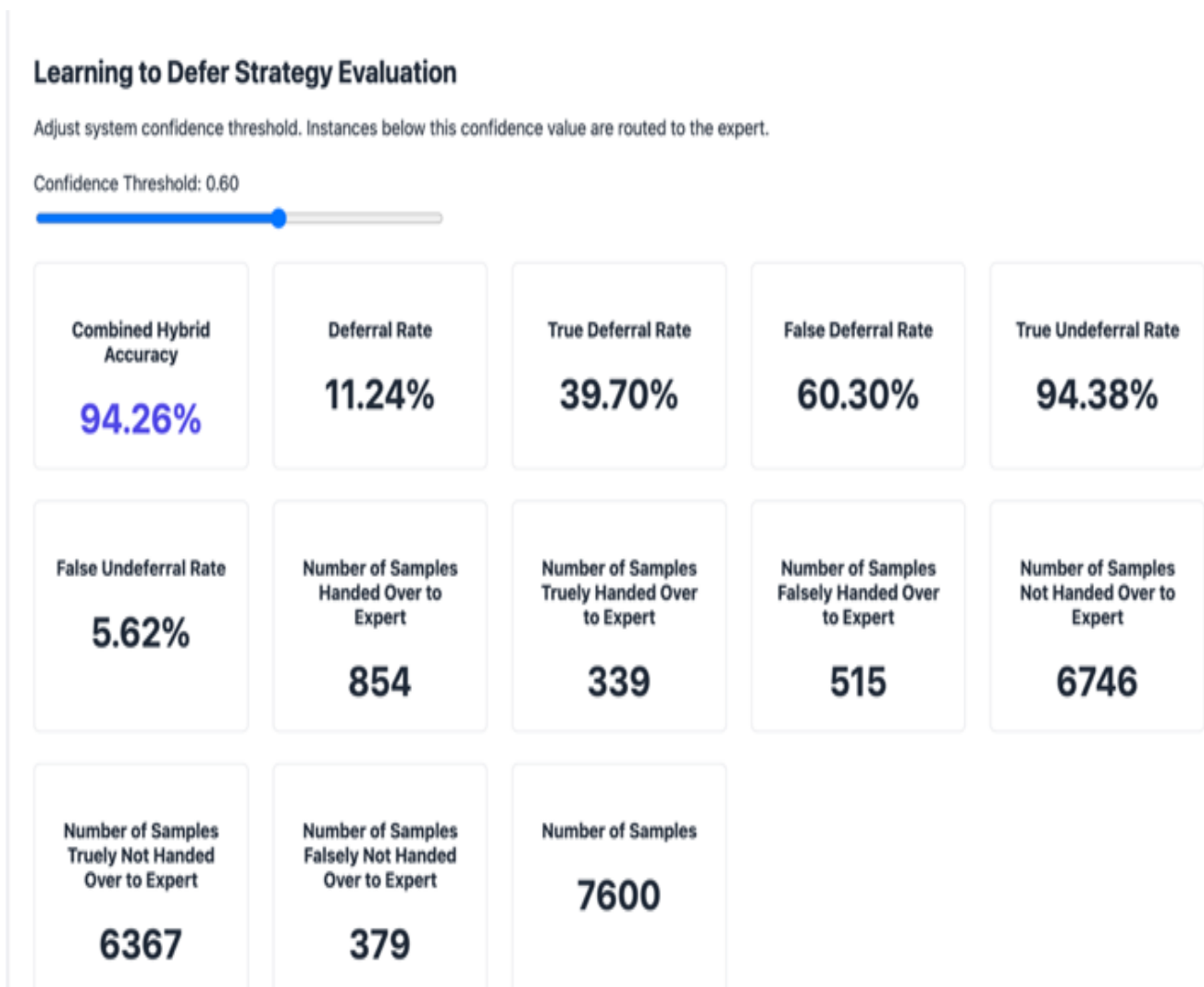


Figure 1: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.60

Learning to Defer Strategy Evaluation

Adjust system confidence threshold. Instances below this confidence value are routed to the expert.

Confidence Threshold: 0.315

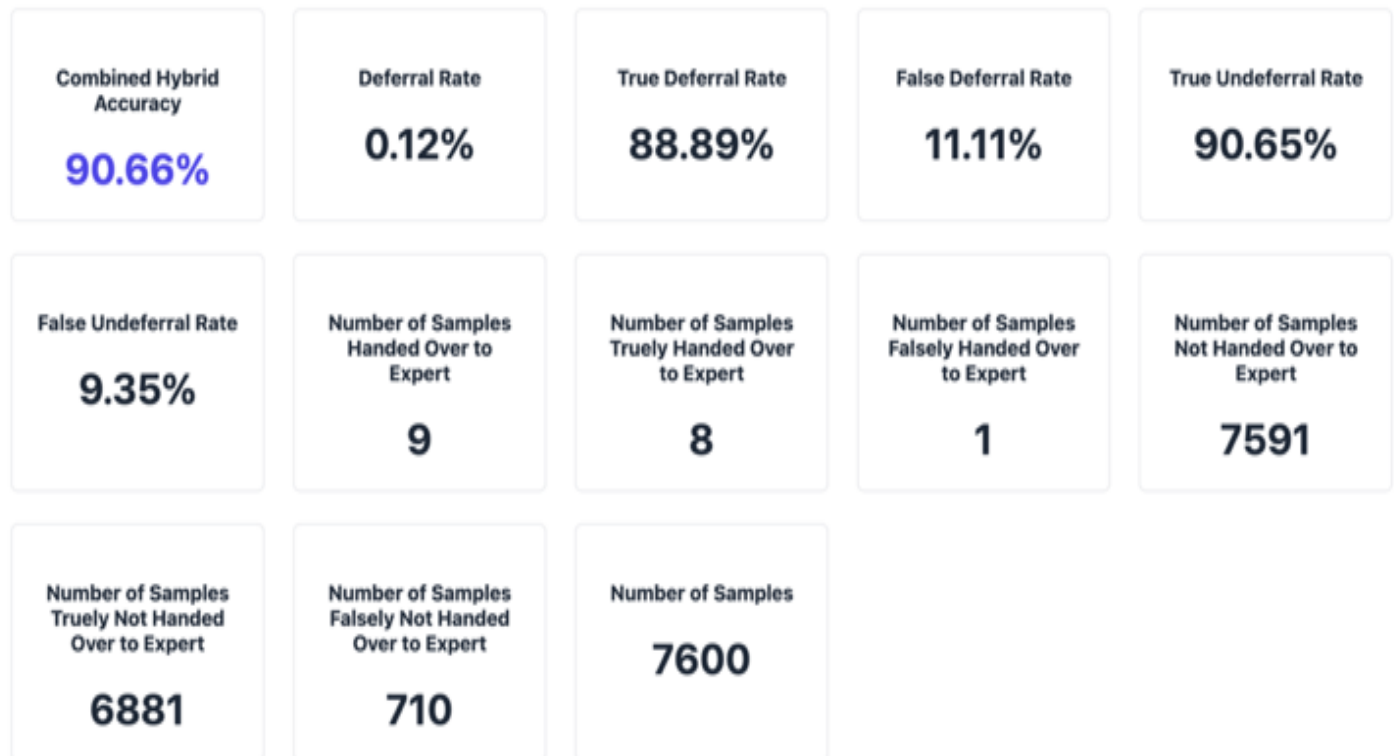


Figure 2: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.315

Learning to Defer Strategy Evaluation

Adjust system confidence threshold. Instances below this confidence value are routed to the expert.

Confidence Threshold: 0.835

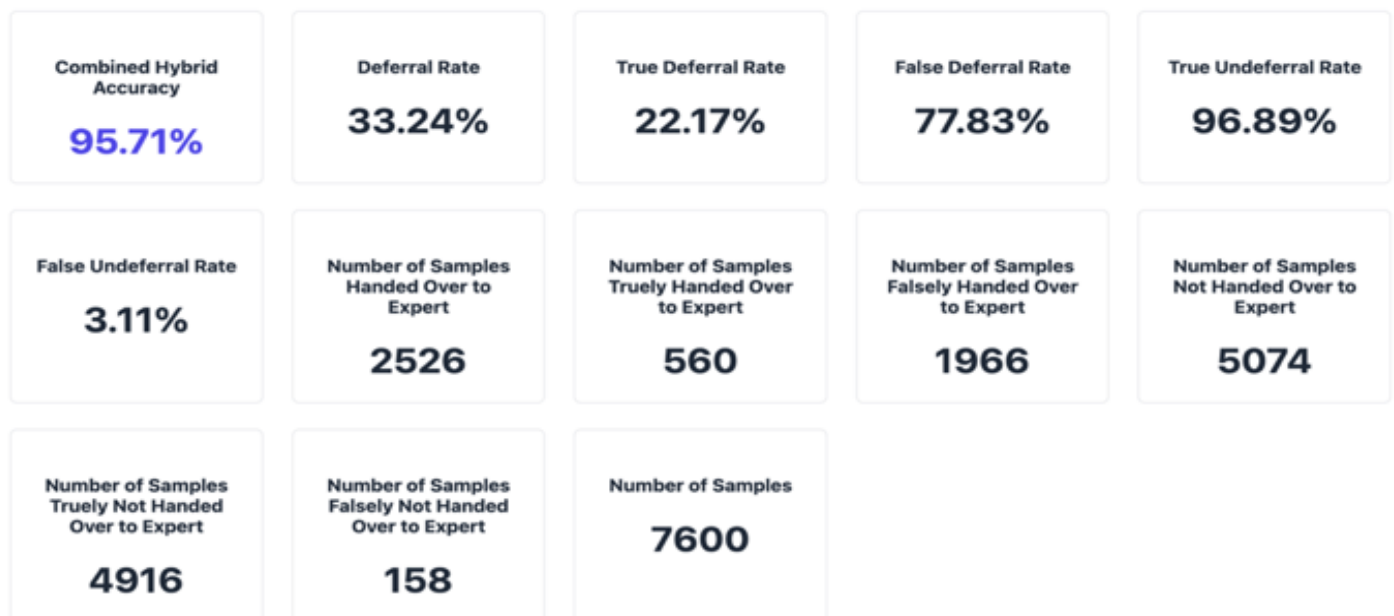


Figure 3: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.835

Learning to Defer Strategy Evaluation

Adjust system confidence threshold. Instances below this confidence value are routed to the expert.

Confidence Threshold: 0.29

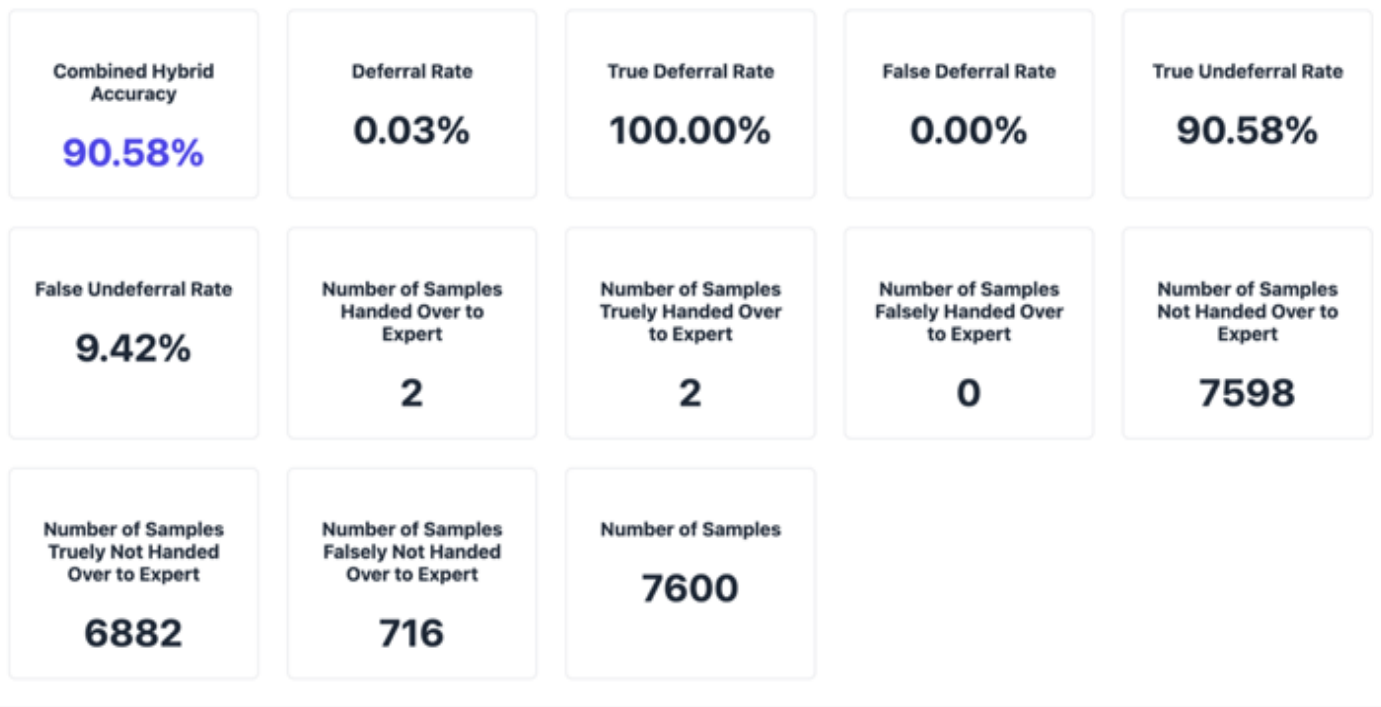


Figure 4: Learning to Defer Strategy Evaluation When Confidence Threshold is 0.29

```
def process_learning_to_defer(self, threshold):
    """Dispatches low-confidence choices to human or expert pipelines."""
    probs = self.baseline_model.predict_proba(self.X_test)
    max_probs = np.max(probs, axis=1)
    classifier_test_preds = np.argmax(probs, axis=1)

    final_predictions = []

    deferral_count = 0
    undefferal_count = 0

    true_deferral_count = 0
    false_deferral_count = 0

    true_undefferal_count = 0
    false_undefferal_count = 0

    for i in range(len(self.X_test)):
        is_classifier_correct = (classifier_test_preds[i] == self.y_test[i])

        if max_probs[i] < threshold:
            final_predictions.append(self.expert_test_preds[i])
            deferral_count += 1

            if not is_classifier_correct:
                true_deferral_count += 1
            else:
                false_deferral_count += 1

        else:
            final_predictions.append(classifier_test_preds[i])
            undefferal_count += 1

            if is_classifier_correct:
                true_undefferal_count += 1
            else:
                false_undefferal_count += 1

    system_accuracy = float(accuracy_score(self.y_test, final_predictions))
    denom_deferral = true_deferral_count + false_deferral_count
    true_deferral_rate = float(true_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
    false_deferral_rate = float(false_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0

    denom_undefferal = true_undefferal_count + false_undefferal_count
    true_undefferal_rate = float(true_undefferal_count / denom_undefferal) if denom_undefferal > 0 else 0.0
    false_undefferal_rate = float(false_undefferal_count / denom_undefferal) if denom_undefferal > 0 else 0.0

    return {
        'system_accuracy': system_accuracy,
        'deferral_rate': float(deferral_count / len(self.X_test))
    }
```

Code Screenshot 5: Code implemented for the learning-to-defer process


```

95         if not is_classifier_correct:
96             true_deferral_count += 1
97         else:
98             false_deferral_count += 1
99
100     else:
101         final_predictions.append(classifier_test_preds[i])
102         undeferral_count += 1
103
104         if is_classifier_correct:
105             true_undeferral_count += 1
106         else:
107             false_undeferral_count += 1
108
109
110
111     system_accuracy = float(accuracy_score(self.y_test, final_predictions))
112     denom_deferral = true_deferral_count + false_deferral_count
113     true_deferral_rate = float(true_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
114     false_deferral_rate = float(false_deferral_count / denom_deferral) if denom_deferral > 0 else 0.0
115
116
117     denom_undeferral = true_undeferral_count + false_undeferral_count
118     true_undeferral_rate = float(true_undeferral_count / denom_undeferral) if denom_undeferral > 0 else 0.0
119     false_undeferral_rate = float(false_undeferral_count / denom_undeferral) if denom_undeferral > 0 else 0.0
120
121     deferral_rate = float(deferral_count / len(self.X_test)) if len(self.X_test) > 0 else 0.0
122
123     return {
124         'system_accuracy': system_accuracy,
125         'deferral_rate': deferral_rate,
126         'true_deferral_rate': true_deferral_rate,
127         'false_deferral_rate': false_deferral_rate,
128         'true_undeferral_rate': true_undeferral_rate,
129         'false_undeferral_rate': false_undeferral_rate,
130         'deferred_count': deferral_count,
131         'undeferral_count': undeferral_count,
132         'true_deferred_count': true_deferral_count,
133         'false_deferred_count': false_deferral_count,
134         'true_undeferral_count': true_undeferral_count,
135         'false_undeferral_count': false_undeferral_count,
136         'total_count': len(self.X_test)
137     }
138

```

Code Screenshot 6: Code implemented for the learning-to-defer process

Here, the system accuracy calculated is a hybrid value. The accuracy value is calculated for the ground truth "y_test" values and final_predictions, which include both the expert predictions and baseline classifier predictions. To avoid a divide-by-zero error in the ratio values, we checked whether the denominator of each ratio value is greater than 0; if so, it is used as it is. Otherwise, the corresponding ratio is set to 0.0.

If the max probability coming from the baseline model trained on the X input test dataset is below the threshold (**max_probs[i] < threshold**), it means that we are deferring to the expert. However, in this scenario, if the baseline classifier is correct for the current index, it is a **false deferral (false positive)** of knowledge to the expert. Otherwise, it is a **true deferral (true positive)** case.

If the max probability coming from the baseline model trained on the X input test dataset is above the threshold (**max_probs[i] >= threshold**), it means that we do not defer to the expert, meaning the baseline model is confident enough in its prediction. However, in this scenario, if the baseline classifier is correct for the current index, it is a **true undeferral (true negative)** of knowledge to the expert. Otherwise, it is a **false undeferral (false negative)** case.

Task 4 & Task 5:

```
project3 > active_learning_evaluation.py > ...

1  from ml_backbone import ml_manager
2
3
4  initial_disagreement = ml_manager.get_disagreement_metrics()
5  print(f"Baseline Model Test Accuracy: {ml_manager.baseline_acc:.4f}")
6  print(f"Simulated Expert Test Accuracy: {ml_manager.expert_acc:.4f}")
7  print(f"Initial Deferral Agreement Rate: {initial_disagreement['agreement_rate']:.4f}")
8  print(f"Initial Deferral Disagreement Rate: {initial_disagreement['disagreement_rate']:.4f}\n")
9
10
11 # Active Learning Simulation Loop
12 print("Starting Active Learning Queries ...")
13 num_queries_to_run = 30
14 print(f"Number of active learning queries to run: {num_queries_to_run}")
15
16 initial_al_accuracy = ml_manager.accuracy_history[0]
17 print(f"The active learning model's test accuracy at Query 0 (Initialization) is: {initial_al_accuracy:.4f}\n")
18
19 for i in range(num_queries_to_run):
20     # Get the next most uncertain sample
21     sample_data = ml_manager.get_next_uncertain_sample()
22     if sample_data is None:
23         break
24
25     # Pass the expert's label into the update system
26     update_result = ml_manager.process_query_update(
27         idx=sample_data['sample_index'],
28         chosen_label=sample_data['simulated_expert_label']
29     )
30
31     # Log the progress
32     print(f"Query {i+1}/{num_queries_to_run} | "
33         f"Test Accuracy: {update_result['current_test_accuracy']:.4f} | "
34         f"Accuracy Gain/Loss Per Query: {update_result['accuracy_gain_per_query']:.4f} | "
35         f"Remaining Deferral Ops: {update_result['total_deferral_opportunities']}\n")
36
37 # Final Visualization of the Active Learning Process
38 print("\nPlotting results...")
39 ml_manager.plot_metrics()
```

Code Screenshot 7: Code implemented for the active learning evaluation and test accuracy change graph plotting during the active learning process

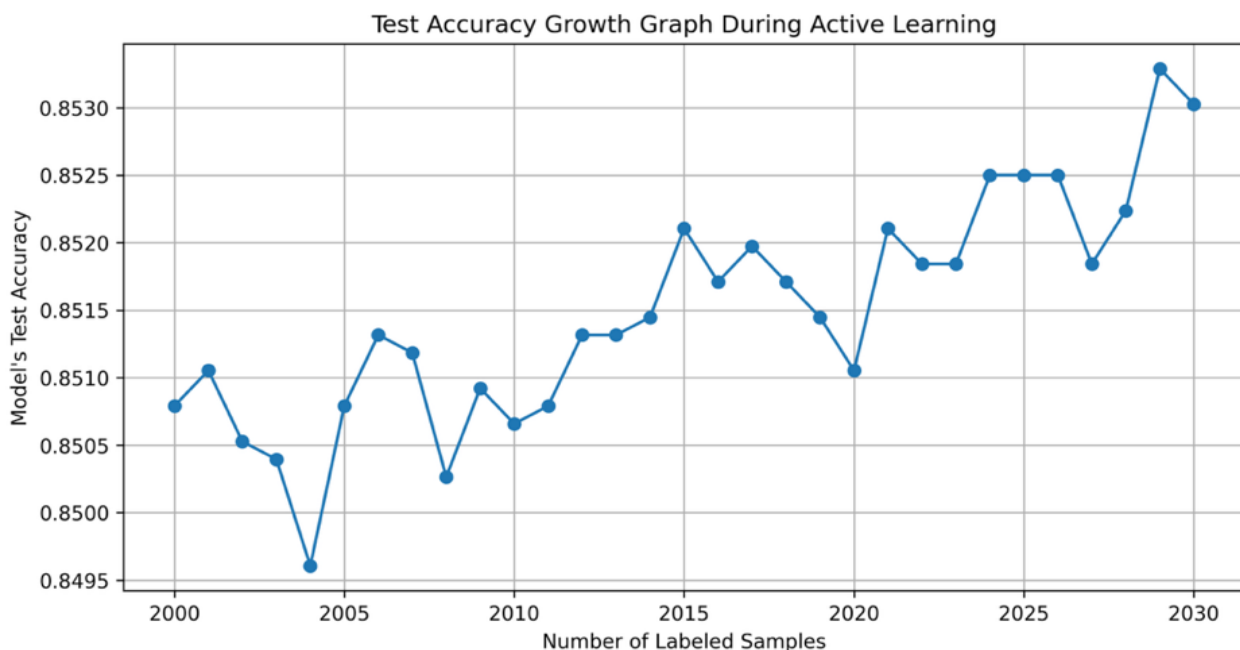


Figure 5: Number of Labeled Samples vs Model's Test Accuracy Graph for 30 Active Learning Queries


```

(hcai_env) (base) bariskaplan@Baris-MacBook-Pro HCAI-PBL-Projects % ls
~$object3_Report.docx  hcai_env  manage.py  pdf_files_of_projects  project3
project4               templates
db.sqlite3             home       media      project1              Project3_Report.docx
project5               iris.csv   pbl        project2              Project3_Report.pdf
static
(hcai_env) (base) bariskaplan@Baris-MacBook-Pro HCAI-PBL-Projects % cd project3
(hcai_env) (base) bariskaplan@Baris-MacBook-Pro project3 % ls
__init__.py            active_learning_evaluation.py  migrations  README.md
__urls.py              __pycache__                  admin.py    ml_backbone.py       requirements.txt
views.py               accuracy_growth_graph.png     apps.py     models.py             tests.py
(hcai_env) (base) bariskaplan@Baris-MacBook-Pro project3 % python active_learning_evaluation.py
Number of news in the training dataset: 120000
Number of news in the testing dataset: 7600
Baseline Model Test Accuracy: 0.9055
Simulated Expert Test Accuracy: 0.9270
Initial Deferral Agreement Rate: 0.8626
Initial Deferral Disagreement Rate: 0.1374

Starting Active Learning Queries ...
Number of active learning queries to run: 30
The active learning model's test accuracy at Query 0 (Initialization) is: 0.8508

Query 1/30 | Test Accuracy: 0.8511 | Accuracy Gain/Loss Per Query: 0.0003 | Remaining Deferral Ops: 1042
Query 2/30 | Test Accuracy: 0.8505 | Accuracy Gain/Loss Per Query: -0.0005 | Remaining Deferral Ops: 1046
Query 3/30 | Test Accuracy: 0.8504 | Accuracy Gain/Loss Per Query: -0.0001 | Remaining Deferral Ops: 1046
Query 4/30 | Test Accuracy: 0.8496 | Accuracy Gain/Loss Per Query: -0.0008 | Remaining Deferral Ops: 1052
Query 5/30 | Test Accuracy: 0.8508 | Accuracy Gain/Loss Per Query: 0.0012 | Remaining Deferral Ops: 1043
Query 6/30 | Test Accuracy: 0.8513 | Accuracy Gain/Loss Per Query: 0.0005 | Remaining Deferral Ops: 1041
Query 7/30 | Test Accuracy: 0.8512 | Accuracy Gain/Loss Per Query: -0.0001 | Remaining Deferral Ops: 1041
Query 8/30 | Test Accuracy: 0.8503 | Accuracy Gain/Loss Per Query: -0.0009 | Remaining Deferral Ops: 1047
Query 9/30 | Test Accuracy: 0.8509 | Accuracy Gain/Loss Per Query: 0.0007 | Remaining Deferral Ops: 1044
Query 10/30 | Test Accuracy: 0.8507 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1046
Query 11/30 | Test Accuracy: 0.8508 | Accuracy Gain/Loss Per Query: 0.0001 | Remaining Deferral Ops: 1045
Query 12/30 | Test Accuracy: 0.8513 | Accuracy Gain/Loss Per Query: 0.0005 | Remaining Deferral Ops: 1041
Query 13/30 | Test Accuracy: 0.8513 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1041
Query 14/30 | Test Accuracy: 0.8514 | Accuracy Gain/Loss Per Query: 0.0001 | Remaining Deferral Ops: 1040
Query 15/30 | Test Accuracy: 0.8521 | Accuracy Gain/Loss Per Query: 0.0007 | Remaining Deferral Ops: 1035
Query 16/30 | Test Accuracy: 0.8517 | Accuracy Gain/Loss Per Query: -0.0004 | Remaining Deferral Ops: 1038
Query 17/30 | Test Accuracy: 0.8520 | Accuracy Gain/Loss Per Query: 0.0003 | Remaining Deferral Ops: 1036
Query 18/30 | Test Accuracy: 0.8517 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1038
Query 19/30 | Test Accuracy: 0.8514 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1040
Query 20/30 | Test Accuracy: 0.8511 | Accuracy Gain/Loss Per Query: -0.0004 | Remaining Deferral Ops: 1040
Query 21/30 | Test Accuracy: 0.8521 | Accuracy Gain/Loss Per Query: 0.0011 | Remaining Deferral Ops: 1036
Query 22/30 | Test Accuracy: 0.8518 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1038
Query 23/30 | Test Accuracy: 0.8518 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1038
Query 24/30 | Test Accuracy: 0.8525 | Accuracy Gain/Loss Per Query: 0.0007 | Remaining Deferral Ops: 1033
Query 25/30 | Test Accuracy: 0.8525 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1034
Query 26/30 | Test Accuracy: 0.8525 | Accuracy Gain/Loss Per Query: 0.0000 | Remaining Deferral Ops: 1034
Query 27/30 | Test Accuracy: 0.8518 | Accuracy Gain/Loss Per Query: -0.0007 | Remaining Deferral Ops: 1037
Query 28/30 | Test Accuracy: 0.8522 | Accuracy Gain/Loss Per Query: 0.0004 | Remaining Deferral Ops: 1036
Query 29/30 | Test Accuracy: 0.8533 | Accuracy Gain/Loss Per Query: 0.0011 | Remaining Deferral Ops: 1030
Query 30/30 | Test Accuracy: 0.8530 | Accuracy Gain/Loss Per Query: -0.0003 | Remaining Deferral Ops: 1031

Plotting results...
Plot successfully saved to: accuracy_growth_graph.png
(hcai_env) (base) bariskaplan@Baris-MacBook-Pro project3 % █

```

Figure 6: Active Learning Evaluation Results and Progress for 30 Active Learning Queries

```

140     def get_next_uncertain_sample(self):
141         """Selects data instances with lowest prediction confidence scores."""
142         if not self.AL_pool_indices:
143             return None
144
145         pool_probs = self.active_learning_model.predict_proba(self.X_train[self.AL_pool_indices])
146         max_pool_probs = np.max(pool_probs, axis=1)
147
148         uncertain_pool_idx = np.argmin(max_pool_probs)
149         global_idx = self.AL_pool_indices[uncertain_pool_idx]
150
151         simulated_label = self.simulate_expert_predict([self.y_train[global_idx]])[0]
152
153         return {
154             'sample_index': int(global_idx),
155             'text': self.X_train_raw[global_idx],
156             'true_label': int(self.y_train[global_idx]),
157             'simulated_expert_label': int(simulated_label),
158             'categories': self.categories
159         }
160
161     def process_query_update(self, idx, chosen_label):
162         """Absorbs feedback loop entries and updates the active learner."""
163         if idx in self.AL_pool_indices:
164             self.AL_pool_indices.remove(idx)
165         if idx not in self.AL_labeled_indices:
166             self.AL_labeled_indices.append(idx)
167
168
169         self.y_train[idx] = chosen_label
170
171         # Retrain dynamic tracker model instance
172         X_train = self.X_train[self.AL_labeled_indices]
173         y_train = self.y_train[self.AL_labeled_indices]
174         self.active_learning_model.fit(X_train, y_train)
175
176         al_preds = self.active_learning_model.predict(self.X_test)
177         current_al_acc = float(accuracy_score(self.y_test, al_preds))
178         self.accuracy_history.append(current_al_acc)
179         self.query_history.append(len(self.AL_labeled_indices))
180
181         # Get the new disagreement metrics after retraining
182         disagreement_data = self.get_disagreement_metrics()
183
184         return {
185             'status': 'success',
186             'current_test_accuracy': current_al_acc,
187             'total_labeled_count': len(self.AL_labeled_indices),
188             'disagreement_rate': disagreement_data['disagreement rate'].

```

Code Screenshot 8: Code implemented for the active learning process and evaluation

```

161 def process_query_update(self, idx, chosen_label):
178     self.accuracy_history.append(current_al_acc)
179     self.query_history.append(len(self.AL_labeled_indices))
180
181     # Get the new disagreement metrics after retraining
182     disagreement_data = self.get_disagreement_metrics()
183
184     return {
185         'status': 'success',
186         'current_test_accuracy': current_al_acc,
187         'total_labeled_count': len(self.AL_labeled_indices),
188         'disagreement_rate': disagreement_data['disagreement_rate'],
189         'total_deferral_opportunities': disagreement_data['total_deferral_opportunities'],
190         'accuracy_gain_per_query': (current_al_acc - self.accuracy_history[-2]) if len(self.accuracy_history) > 1 else 0
191     }
192
193 def plot_metrics(self, save_path="accuracy_growth_graph.png"):
194     plt.figure(figsize=(10, 5))
195     plt.plot(self.query_history, self.accuracy_history, marker='o')
196     plt.title("Test Accuracy Growth Graph During Active Learning")
197     plt.xlabel("Number of Labeled Samples")
198     plt.ylabel("Model's Test Accuracy")
199     plt.grid(True)
200
201     # Save the plot first
202     plt.savefig(save_path, dpi=300, bbox_inches='tight')
203     print(f"Plot successfully saved to: {save_path}")
204
205     # Then, display the plot
206     plt.show()
207
208 def get_disagreement_metrics(self):
209     """Measures how often the active learning model is wrong while the simulated expert is right."""
210     model_preds = self.active_learning_model.predict(self.X_test)
211     model_wrong = (model_preds != self.y_test)
212     expert_right = (self.expert_test_preds == self.y_test)
213
214     # The model is wrong and the expert is right
215     disagreement_mask = model_wrong & expert_right
216     disagreement_rate = np.mean(disagreement_mask)
217     agreement_rate = 1 - disagreement_rate
218
219     return {
220         'agreement_rate': float(agreement_rate),
221         'disagreement_rate': float(disagreement_rate),
222         'total_deferral_opportunities': int(np.sum(disagreement_mask)),
223         'initial_model_accuracy': float(self.accuracy_history[0])
224     }
225

```

Code Screenshot 9: Code implemented for the active learning process and evaluation

Active Learning & Interactive Human Expert Loop

The system uses **Least Confidence Sampling** to detect items it does not understand. Query the simulated expert or choose the correct label yourself as the Human Expert.

Current Uncertain Sample Pool Item:

BA boss admits 'we got it wrong' British Airways chief executive Rod Eddington admits the airline "got it wrong" after staff shortages led to travel chaos.

Simulated Expert's Decision Recommendation:

World

Query Simulated Expert

OR Choose As Human Expert

Class 0: World

Class 1: Sports

Class 2: Business

Class 3: Sci/Tech

Active Learning Model Tracking:

Active Learner Model Test Accuracy

85.08 %

Number of Manual Active Learning Queries by User

0

Total Active Learning Queries Collected

2000 (Number of Initially Labeled and Trained Seeds: 2000)

Generate Test Accuracy Growth Graph

Clear Test Accuracy Growth Graph

Figure 7: Initial Active Learning Interface State

Active Learning & Interactive Human Expert Loop

The system uses **Least Confidence Sampling** to detect items it does not understand. Query the simulated expert or choose the correct label yourself as the Human Expert.

Current Uncertain Sample Pool Item:

Fox Orders Pilot Starring Barenaked Ladies LOS ANGELES - Fox Broadcasting Co. has ordered a pilot for a variety show starring the Barenaked Ladies that will feature the playful rock group performing music and comedy skits along with guest actors...

Simulated Expert's Decision Recommendation:

World

Query Simulated Expert

OR Choose As Human Expert

Class 0: World

Class 1: Sports

Class 2: Business

Class 3: Sci/Tech

Active Learning Model Tracking:

Active Learner Model Test Accuracy

85.18%

Number of Manual Active Learning Queries by User

7

Total Active Learning Queries Collected

2007 (Number of Initially Labeled and Trained Seeds: 2000)

Generate Test Accuracy Growth Graph

Clear Test Accuracy Growth Graph

Figure 8: Active Learning Interface State After 7 User Queries

Active Learning & Interactive Human Expert Loop

The system uses **Least Confidence Sampling** to detect items it does not understand. Query the simulated expert or choose the correct label yourself as the Human Expert.

Current Uncertain Sample Pool Item:

Renewable energy may be offered by Empire Electric #39;s 15,000 member-owners may be able to get all or part of their electricity from renewable sources starting in January.

Simulated Expert's Decision Recommendation:

Sci/Tech

Query Simulated Expert

OR Choose As Human Expert

Class 0: World

Class 1: Sports

Class 2: Business

Class 3: Sci/Tech

Active Learning Model Tracking:

Active Learner Model Test Accuracy

85.20%

Number of Manual Active Learning Queries by User

25

Total Active Learning Queries Collected

2025 (Number of Initially Labeled and Trained Seeds: 2000)

Generate Test Accuracy Growth Graph

Clear Test Accuracy Growth Graph

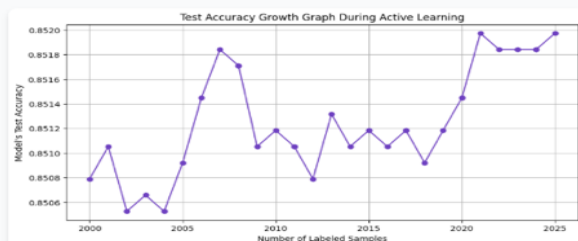


Figure 9: Active Learning Interface State After Generating Test Accuracy Growth Graph Up to 25 User Queries

Justifications of the Design and Implementation Choices: In the active learning process, it is aimed to actively choose which text instance(s) should be presented/deferred to the expert in order to efficiently learn the expert's competence area, the text classification task, and the case where the deferral of knowledge is beneficial, given no access to any expert label during training. In order to achieve this, I have used a sampling strategy named "**Uncertainty-Based (Least-Confidence) Sampling**" in the "**get_next_uncertain_sample()**" function. I have predicted the probability distribution for the unlabeled part of the X training data corresponding to the active learning pool indices. Then, I have selected the maximum value in each row of this probability distribution (using the "**np.argmax(pool_probs, axis=1)**" call) and created a maximum pool probabilities array. At this step, the benefit of deferral needs to be somehow increased. I have achieved this by selecting the global index where the active learning model is the least confident. Here, the least confident index from the pool is obtained by selecting the minimum index value (using the "**np.argmin(values)**" call) from the maximum pool probabilities array created. This suggests that the deferral is more useful because of the lower model confidence at that index.

In the "**process_query_update**" function, I am updating the AL_pool_indices (**pool indices for active learning**) and AL_labeled_indices (**labeled indices for active learning**) lists after labeling each text instance. When I label a text sample, I remove its index from the active learning pool indices and uniquely add it to the labeled indices. Then, I retrain the active learning model with the new list of labeled indices for active learning. After that, I am evaluating the retrained model to get the recent accuracy value of the active learning model trained with a new query.

Initially, the number of labeled and trained seed values is 2000. When we look at the test accuracy growth graph **for 25 new queries, given in Figure 5**, we can observe that the overall trend of test accuracy is upward as new queries are provided. However, when an active learner queries highly borderline samples, the decision boundary of the active learning classifier model aggressively changes, which often leads to performance stagnation or decreases on the test set before the self-correction and increase in accuracy. As the user provides new queries, the test accuracy seems to be growing slowly, which is reasonable given the **large size of the training dataset**, which is **120000**.

I have kept track of the current test accuracy scores of the active learner in *self.accuracy_history* and the number of labeled samples in *self.query_history*. In addition to the graph plotted, I have used deferral agreement & disagreement rates, current test accuracy of the active learning model, and accuracy gain/loss per user query for evaluating the active learning model. They indicate high initial (**0.8508**) & last (**0.8530**) test accuracies, which means active learning performs well on the unseen test data. In order to calculate the disagreement rate between the active learning model and the simulated expert, I have first calculated the part where **the active learning model predictions on the unseen 'y_test' data do not match the ground truth 'y_test' data**. Then, I have calculated the part where **the simulated expert predictions on the unseen 'y_test' data do match the ground truth 'y_test' data**. After that, I get the "**AND**" mask of these parts. Finally, I have applied *np.mean()* on the created mask, which treats the true boolean value as 1 and the false boolean value as 0. The *np.mean()* call will then give the deferral disagreement rate, which indicates how often the active learning model is wrong while the simulated expert is right.